

Git and Version Control

Tracking changes, branching, collaborating and recovering from mistakes

In this guide

- Introduction
- 1. Core Concepts and Setup
- 2. The Everyday Workflow
- 3. Branching and Merging
- 4. Working with Remote Repositories
- 5. Undoing Changes and Inspecting History
- 6. Team Workflows and Best Practices
- Conclusion

Introduction

Version control is a system that records changes to files over time so you can recall specific versions later, compare them, and work with other people without overwriting each other's work. **Git** is the most popular version control system in the world. It is free, fast, distributed, and the foundation of platforms such as GitHub, GitLab and Bitbucket.

Without version control, projects accumulate files named `report_final_v2 REALLY_final.docx`, and collaboration means emailing copies. With Git, every change is a recorded snapshot with an author, a timestamp and a message, and you can travel back to any earlier state in seconds.

Why use Git?

- **History:** see who changed what, when and why.
- **Safety:** undo mistakes and recover deleted work.
- **Branching:** experiment on the side without disturbing stable code.
- **Collaboration:** many people can work on the same project in parallel.
- **Distributed:** every clone contains the full history, so you can work offline.

What you will learn

- 1 The key concepts: repository, commit, staging area and branch.
- 2 The everyday workflow: init, add, commit, status and log.
- 3 Branching and merging, including resolving conflicts.
- 4 Working with remotes: clone, push, pull and pull requests.
- 5 How to undo changes safely.
- 6 Team workflows and good habits.

Git vs GitHub: Git is the tool that runs on your computer. GitHub is an online service that hosts Git repositories and adds features such as pull requests and issue tracking.

1. Core Concepts and Setup

A **repository** (repo) is a project folder whose history Git tracks; the data lives in a hidden `.git` directory. History is stored as a series of **commits**. Each commit is a snapshot of the project plus a message, an author and a pointer to its parent commit, forming a chain that can also branch and merge.

The three areas

Area	Purpose
Working directory	The actual files you edit
Staging area (index)	A holding area where you choose which changes go into the next commit
Repository	The permanent history of commits

The staging area lets you build clean, focused commits. You may edit five files but stage and commit only the two that belong to one logical change.

First-time setup

```
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
git --version
```

Key vocabulary

Term	Meaning
Commit	A saved snapshot with a message
Branch	A movable pointer to a line of commits
HEAD	Pointer to the commit or branch you are currently on
Remote	A copy of the repo hosted elsewhere, such as <code>origin</code>
Merge	Combining the work of two branches
Clone	A full copy of a remote repository

Key points

- A commit is a snapshot, not just a list of differences.
- Files move from working directory to staging area to repository.
- Configure your name and email once per machine.

2. The Everyday Workflow

Most of your time with Git uses a small loop: edit files, review the changes, stage them, and commit. Start a new project with `git init`, or copy an existing one with `git clone`.

```
mkdir myproject && cd myproject
git init                      # create a repository
echo "# My Project" > README.md
git status                    # what changed?
git add README.md             # stage the file
git commit -m "Add README"    # save a snapshot
```

The essential commands

Command	What it does
<code>git status</code>	Shows changed, staged and untracked files
<code>git add <file></code>	Stages a file (<code>git add .</code> stages everything)
<code>git commit -m "msg"</code>	Records the staged changes
<code>git diff</code>	Shows unstaged changes; <code>--staged</code> shows staged ones
<code>git log --oneline</code>	Compact history of commits
<code>git show <commit></code>	Details of a single commit

```
git log --oneline --graph --decorate -5
# 3f2a1c9 (HEAD -> main) Add login form
# a81d0be Fix typo in README
# 5c9e77f Add README
```

Ignoring files

Some files should never be tracked: passwords, virtual environments, build output, editor settings. List patterns in a `.gitignore` file at the project root.

```
# .gitignore
.venv/
__pycache__/
*.log
.env
node_modules/
```

Writing good commit messages

- Use a short summary line (about 50 characters) in the imperative mood: 'Add search filter'.
- Explain **why** in the body if the reason is not obvious.
- Make each commit one logical change so it can be reviewed and reverted on its own.

Never commit secrets: Passwords, API keys and tokens committed to a repository stay in its history even after deletion. Keep them in environment variables or a secrets manager.

3. Branching and Merging

A **branch** is a lightweight, independent line of development. Creating one costs almost nothing, so Git encourages a habit of doing every feature or fix on its own branch and keeping **main** stable.

```
git branch                # list branches
git switch -c feature/search # create and switch to a new branch
# ...edit files, then...
git add .
git commit -m "Add search box"
git switch main           # go back to main
git merge feature/search  # bring the work in
git branch -d feature/search # delete the finished branch
```

Fast-forward and three-way merges

If **main** has not moved since you branched, Git simply slides the pointer forward (a **fast-forward**). If both branches have new commits, Git creates a **merge commit** that has two parents and combines the histories.

Resolving merge conflicts

A conflict occurs when both branches changed the same lines. Git cannot decide which version to keep, so it pauses and marks the file:

```
<<<<<<< HEAD
title = "Welcome"
=====
title = "Hello there"
>>>>>>> feature/search
```

- 1 Open the file and decide what the final text should be (one side, the other, or a mix).
- 2 Delete the marker lines (<<<<<<<, =====, >>>>>>>).
- 3 Run `git add <file>` to mark it resolved.
- 4 Run `git commit` to complete the merge (or `git merge --abort` to cancel).

Rebase: an alternative to merge

`git rebase main` replays your branch's commits on top of the latest **main**, producing a straight, linear history. It is tidy but rewrites commits, so never rebase branches that others have already pulled.

Key points

- Branch early and often; delete branches after merging.
- Conflicts are normal and simple to fix once you know the markers.

- Merge preserves history; rebase makes it linear but rewrites it.

4. Working with Remote Repositories

A **remote** is a version of your repository hosted on a server such as GitHub. Remotes allow backup and collaboration. By convention the main remote is named **origin**.

```
git clone https://github.com/user/project.git      # copy a remote repo
git remote -v                                     # list remotes
git remote add origin https://github.com/user/project.git
git push -u origin main                           # upload commits (first time)
```

Sharing and updating

Command	Purpose
<code>git fetch</code>	Download new commits without changing your files
<code>git pull</code>	Fetch and merge remote changes into your branch
<code>git push</code>	Upload your local commits to the remote
<code>git push -u origin <branch></code>	Publish a new branch and link it to the remote

```
git pull                                           # start the day with the latest code
git switch -c fix/login-bug
# ...work and commit...
git push -u origin fix/login-bug
```

If a push is rejected, someone else has pushed first. Run `git pull`, resolve any conflicts, and push again.

Pull requests

On hosting platforms, you propose merging your branch through a **pull request** (called a merge request on GitLab). Teammates review the changes, leave comments, automated tests run, and once approved the branch is merged. It is the standard way to keep quality high.

Authentication

- **HTTPS with a personal access token** is easy to set up.
- **SSH keys** (`ssh-keygen -t ed25519`) avoid typing credentials repeatedly.

Forks

To contribute to a project you do not own, **fork** it on the hosting site (your own copy), clone the fork, push a branch to it, and open a pull request back to the original.

5. Undoing Changes and Inspecting History

Git's greatest strength is that almost nothing is truly lost. Knowing which undo tool to use depends on where the change currently lives.

Situation	Command
Discard edits to a file (not staged)	<code>git restore <file></code>
Unstage a file but keep the edits	<code>git restore --staged <file></code>
Fix the last commit message or add a forgotten file	<code>git commit --amend</code>
Undo a pushed commit safely	<code>git revert <commit></code>
Move the branch back (local only)	<code>git reset --hard <commit></code>

revert versus reset

`git revert` creates a **new commit** that reverses an earlier one, leaving history intact. It is the right choice for shared branches. `git reset` moves the branch pointer backwards and can discard work; use it only on local commits you have not shared.

Danger: `git reset --hard` and `git clean -fd` permanently discard uncommitted changes. Check `git status` first.

Stashing

`git stash` shelves uncommitted changes so you can switch tasks with a clean directory, then bring them back.

```
git stash          # save work in progress
git switch main
# ...do something urgent...
git switch feature/search
git stash pop      # restore the shelved changes
```

Investigating history

```
git log --author="Ira" --since="2 weeks ago" --oneline
git blame app.py          # who last changed each line
git diff main..feature/search # differences between branches
git bisect start           # binary-search for the breaking commit
```

The reflog: your safety net

`git reflog` records every place HEAD has pointed recently. Even after an accidental reset or deleted branch, you can usually find the lost commit's id and recover it with `git switch -c rescue <id>`.

6. Team Workflows and Best Practices

Git does not force a particular process, so teams agree on a workflow. The best choice depends on team size and release style.

Workflow	How it works	Good for
Feature branch	Short-lived branch per feature, merged via pull request	Most teams
Trunk-based	Tiny changes merged into <code>main</code> many times a day, guarded by tests	Fast CI/CD teams
Git Flow	Long-lived <code>develop</code> and release branches	Scheduled releases
Forking	Contributors work in forks and open pull requests	Open source

Habits of effective Git users

- 1 Commit small and often, with clear messages.
- 2 Pull before you start and before you push.
- 3 Keep `main` always working; protect it with required reviews and automated tests.
- 4 Use descriptive branch names such as `feature/payment-form` or `fix/null-error`.
- 5 Review your own diff (`git diff --staged`) before committing.
- 6 Tag releases so versions are easy to find.

```
git tag -a v1.0.0 -m "First stable release"
git push origin v1.0.0
```

Common pitfalls

- Committing large binary files, secrets or generated output.
- Working directly on `main` and mixing unrelated changes in one commit.
- Force-pushing (`git push --force`) to shared branches, which overwrites teammates' work.
- Ignoring conflicts until they become huge; merge from `main` regularly.

Quick command cheat sheet

```
git init | git clone <url>      start
git status | git diff | git log  inspect
git add . && git commit -m "msg"  save
git switch -c <branch>          branch
git merge <branch>              combine
git pull | git push              sync
git restore <file> | git revert  undo
```

Conclusion

Git gives you a complete, reliable memory for your projects and a structured way to collaborate. You have learned the concepts of repositories, commits and the staging area, the daily edit-add-commit cycle, how to isolate work in branches and merge it back, how to share via remotes and pull requests, and how to undo mistakes without panic.

Key takeaways

- Commit small, meaningful snapshots with clear messages.
- Use branches for every feature or fix and keep `main` stable.
- Fetch or pull often; resolve conflicts calmly by editing the marked files.
- Prefer `revert` for shared history and reserve `reset` for local work.
- Never commit secrets, and never force-push to shared branches.

Suggested next steps

- 1 Create a GitHub account and push a practice project.
- 2 Make a pull request on a small open-source repository tagged 'good first issue'.
- 3 Learn interactive rebase (`git rebase -i`) to tidy commits before review.
- 4 Explore CI tools such as GitHub Actions to run tests automatically on every push.

Remember: If you ever feel you have broken a repository, stop and run `git status` and `git reflog`. Committed work is almost always recoverable.